

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ABES ENGINEERING COLLEGE, GHAZIABAD

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: AI Based Resume Screening and Candidate Shortlisting System

Standard Academic Compliance: IEEE Std 830-1998

Project Domain: Artificial Intelligence and Machine Learning (AIML)

SUBMITTED BY:

1. Vansh Singhal (Roll No: 2400321540210)

Class / Section: CSE-DS-B

Degree: Bachelor of Technology (B.Tech)

Academic Year: 2026 - 2027

UNDER THE GUIDANCE OF:

Prof. / Dr.- Aaditya

Designation / Title: HCL Tech Trainer

Department of Computer Science & Engineering

Status: 3rd Year Project Specification

Date of Submission: 01-10-2026

Document Revision & Approval History

0.1	28-Sep-2026	Initial project scope, dataset and ML workflow definition	Vansh Singhal	Project Guide
1.0	01-Oct-2026	Final SRS baseline updated with role-based screening and deployed architecture	Vansh Singhal	HOD / Committee

Guidance Note: This Software Requirements Specification document is intended for 3rd-year university project evaluation, project review, implementation, testing, and final demonstration. The section sequence and academic sign-off structure follow the provided B.Tech SRS template.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional requirements, interfaces, performance expectations, constraints, and design context of the **“AI Based Resume Screening and Candidate Shortlisting System”**. The system accepts a PDF resume and a selected job role, automatically obtains the role-specific requirements, performs resume analysis, and produces an interpretable final ATS-style score with a shortlist decision.

1.2 Scope of the System

The proposed application automates the initial screening of resumes by combining PDF text extraction, explicit skill matching, experience matching, and a trained machine-learning classification pipeline.

- In-Scope Capabilities: PDF resume upload, job-role selection from a predefined role catalogue, automatic retrieval of required skills, required experience and job description, resume text extraction, skill matching, experience matching, ML-based scoring, weighted final ATS score calculation, shortlist decision, matched/missing skill display, and web-based result visualization.
- Out-of-Scope Boundaries: OCR for scanned/image-only resumes, semantic NLP/embeddings, automatic parsing of arbitrary external job descriptions, enterprise HR/ERP integrations, persistent recruitment history beyond the implemented application workflow, and automated employment decisions.
- Expected Benefits: Faster initial screening, consistent comparison against predefined job-role requirements, transparent score components, identification of matched and missing skills, and immediate presentation of a final screening result.

1.3 Definitions, Acronyms, and Abbreviations

Standard	SRS	Software Requirements Specification following the provided IEEE Std 830-1998 academic structure.
Architecture	API	Application Programming Interface used for communication between the React frontend and FastAPI backend.
Machine Learning	ML	Trained machine-learning pipeline used to generate the ML model score for screening.
Document	PDF	Portable Document Format used as the primary resume input.
Network	REST	HTTP-based stateless communication style used by the frontend and backend.
Project Specific	ATS Score	Project-defined weighted screening score combining skill, experience and ML scores.
Project Specific	Job Role	Predefined role selected by the user; its required skills, experience and job description are supplied by the backend.

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- Software Engineering (10th Edition): Ian Sommerville, Pearson Education.
- Project implementation references: FastAPI, React/Vite, scikit-learn, PyMuPDF, Render and Vercel documentation, and the project dataset used for model training.

1.5 Document Overview

The remainder of this SRS describes the product perspective, user classes, operating environment, constraints and assumptions. Section 3 defines functional requirements for job-role selection, resume processing, scoring and result presentation. Section 4 defines external interfaces, Section 5 defines non-functional requirements, and Section 6 lists the system design and analysis models.

2. Overall Description

2.1 Product Perspective

The project is a multi-tier web application comprising a React/Vite presentation layer and a FastAPI application/API layer. The backend extracts text from uploaded PDF resumes, retrieves predefined job-role requirements, performs skill and experience matching, invokes the trained ML pipeline, and

calculates the final ATS score. The production frontend is hosted on Vercel and the backend API is hosted on Render.

[System Context Diagram]

Candidate / Evaluator → React/Vite Frontend → FastAPI Backend → Resume Text Extraction + Job Role Catalogue + ML Model → Screening Result.

The result returned to the frontend contains the final ATS score, shortlist decision, matched skills, missing skills, experience match, ML score, and selected job-role details.

2.2 User Classes and Characteristics

The target software is designed for users who submit or evaluate resumes against predefined job roles. The system does not require specialized client software.

Candidate / Resume Submitter	Basic to Intermediate	Uploads a PDF resume, enters experience and education information, selects a job role, and views the screening result.
Recruiter / Evaluator	Intermediate	Reviews screening scores, matched skills, missing skills, experience match and shortlist decisions.
Guest / Public Visitor	Basic	Can access the public web interface and understand the project workflow.

2.3 Operating Environment

The application operates within the following software and hardware environments:

- Client Requirements: Modern web browser such as Google Chrome, Microsoft Edge, Mozilla Firefox or Safari with a responsive web interface.
- Frontend and Backend Runtime: React/Vite for the frontend and Python/FastAPI for the backend. The deployed frontend runs on Vercel and the deployed backend runs on Render.
- Machine Learning and Document Processing: scikit-learn and joblib for model loading/inference; PyMuPDF for PDF text extraction. The model is trained from the project resume dataset.

2.4 Design and Implementation Constraints

- Academic Budget Constraint: The implementation uses free or academic-tier cloud resources where available, including Vercel and Render.
- Input Constraint: The supported resume input is a text-readable PDF. Scanned/image-only PDFs requiring OCR are outside the current scope.
- Modularity Constraint: Frontend, API logic, job-role configuration and ML model are maintained as separate components so that job roles and model versions can be updated independently.

2.5 Assumptions and Dependencies

- Assumptions: Users have Internet connectivity, provide a valid PDF resume, and enter truthful experience and education information. Job-role requirements are assumed to be maintained by the project configuration.
- Dependencies: Availability of the deployed Vercel frontend, Render backend API, trained model file, and the Python/React runtime dependencies.

3. System Features and Functional Requirements

Requirements are categorized by functional subsystem modules with unique requirement identifiers (FR-x.x):

3.1 Module 1: Resume Input and Job Role Selection

FR-1.1	Resume PDF Upload	Accept a readable PDF resume through the screening form.	High
FR-1.2	Job Role Selection	User selects one predefined job role from the dropdown.	High
FR-1.3	Automatic Requirement Retrieval	Backend retrieves required skills, required experience and job description for the selected role.	High

3.2 Module 2: Resume Processing and ML Screening

FR-2.1	PDF Text Extraction	Extract text from the uploaded PDF using PyMuPDF.	High
FR-2.2	Skill Matching	Normalize and compare resume skills/text against the selected role's required skills.	High
FR-2.3	Experience Matching	Compare candidate experience with the required experience for the selected role.	High

3.3 Module 3: Scoring, Shortlisting and Results

FR-3.1	Final ATS Score	Calculate Skill Match 60%, Experience Match 20% and ML Model Score 20%.	High
--------	-----------------	---	------

FR-3.2	Matched and Missing Skills	Display required skills found in the resume and required skills not found.	High
FR-3.3	Shortlist Decision	Mark candidate Shortlisted when the final ATS score is ≥ 70 ; otherwise Not Shortlisted.	High

4. External Interface Requirements

4.1 User Interfaces (UI)

The interface provides a resume upload control, candidate information fields, a predefined job-role dropdown, role-specific requirement preview, and a result view showing the final ATS score, shortlist decision, skill match, experience match, ML model score, matched skills, missing skills, and selected role information.

[User Interface Wireframe / Screen Reference]

1. Resume Screening Form – PDF upload, experience, education and job-role selection.
2. Job Role Preview – required skills and experience are automatically displayed.
3. Screening Result – final ATS score, shortlist decision and score breakdown.

4.2 Hardware Interfaces

Specification of physical interaction points:

- Standard Deployment: Keyboard, mouse, display and a supported web browser are sufficient for interaction with the application.
- Embedded / IoT Variants: Not applicable to the proposed resume screening system.

4.3 Software Interfaces

- Frontend-to-Backend Interface: React/Vite communicates with the FastAPI backend through HTTP/HTTPS REST API requests and multipart form data for PDF upload.
- Machine Learning and Document Utilities: PyMuPDF performs PDF text extraction; scikit-learn and joblib load and execute the trained ML pipeline. Job-role definitions are maintained in the backend configuration.

4.4 Communication Interfaces

- Web Traffic Protocol: HTTP/HTTPS REST communication between the deployed frontend and backend. Production communication uses HTTPS.
- Data Serialization: API responses are exchanged as JSON; uploaded resumes are transmitted using multipart form data.

5. Non-Functional Requirements (NFRs)

5.1 Performance	Responsive web interface and practical API response for a single resume screening request.	Functional testing with representative PDF resumes and repeated requests.
5.2 Security	Production API communication uses HTTPS; local environment variables are excluded from source control.	Review deployment configuration, .gitignore and browser network requests.
5.3 Reliability	System should return a valid screening result or a clear error for supported PDF inputs.	Test valid PDFs, invalid files, missing inputs and API failures.
5.4 Portability	Application should operate in modern desktop browsers and responsive layouts.	Browser testing using Chrome, Edge, Firefox and responsive viewport inspection.

6. System Design and Analysis Models

University evaluation requires standard graphical analysis models illustrating the dynamic and static behavior of the application. The models for this project should represent the updated role-based workflow: user uploads a resume, selects a job role, backend retrieves role requirements, the ML and rule-based components process the resume, and the result is returned.

Use Case Diagram	Maps Candidate/Evaluator interactions such as upload resume, select job role, screen resume and view result.	UML Standard diagram + Use Case specification.
Data Flow Diagram (DFD)	Shows Resume PDF and candidate inputs flowing through the frontend, FastAPI backend, role catalogue and ML screening components.	Level 0 Context and Level 1 process flow.
Entity-Relationship / Data Model	Represents logical project data such as Job Role, Required Skills, Resume Input and Screening Result.	Crow's Foot or equivalent academic data model.

Activity / Sequence Diagram	Shows the chronological screening flow from role selection and upload to final score and shortlist result.	UML Activity / Sequence diagram.
-----------------------------	--	----------------------------------

7. Academic Review & Sign-Off

This Software Requirements Specification document has been submitted and reviewed by the academic supervisory panel. The implementation described in this document corresponds to the deployed project architecture consisting of a Vercel frontend and Render FastAPI backend.

Signature: _____
Date: _____

Signature: _____
Date: _____

Signature: _____
Date: _____